

Traveling Salesman Problem

von Marc-Michel Münch, geb. 27.01.1991 in Roth

Github Download Section: <https://github.com/Cranjah/Casual-TSP-Iterations>

In diesem Repository (siehe Link oben) werde ich meine Experimente zum P-NP-Problem veröffentlichen (das zu den berühmten Millennium-Problemen gehört und eines der wichtigsten ungelösten Probleme der theoretischen Informatik und der Komplexitätstheorie ist). Insbesondere das Problem des Handlungsreisenden (Traveling Salesman Problem, kurz TSP) als eines der NP-vollständigen Probleme steht im Mittelpunkt meiner Experimente hier. Die Algorithmen sind – zum Zwecke der Experimentierbarkeit und exakter vergleichbarer Berechnungen – in Python 3 implementiert und folgen einem Beispiel für ein TSP der University of Helsinki aus ihrem Online-Kurs „Building AI“ (via Elements of AI), das mich neben meinem dualen Studium der Informatik an der Hochschule für Wirtschaft und Recht Berlin und den Praxisphasen bei der DB Systel GmbH dazu inspiriert hat, einen Teil meiner Freizeit dafür zu nutzen – da ich denke, dass dies auch eine nützliche Vorbereitung für Probleme sein könnte, mit denen ich wahrscheinlich während meiner Arbeit bei der DB Systel GmbH oder allgemein bei einem Eisenbahn- oder Logistikunternehmen konfrontiert sein werde.

Doch worum genau geht es beim P-NP-Problem und worum beim TSP? Kurz beschrieben ist das P-NP-Problem insofern eigentlich "nur" ein Problem der Optimierung für die asymptotische Laufzeit von Algorithmen. Um entsprechende Algorithmen effizient ausführen zu können und nicht in ein erhebliches Laufzeitproblem zu stolpern, haben sich bekanntermaßen Algorithmen etabliert, die in polynomieller Laufzeit – also beispielsweise in $O(n^2)$ oder auch in der Zeit von $O(n^3)$ zu einem Output kommen, also terminieren – man spricht hier von der Komplexitätsklasse P, dabei ist jedoch (fast) egal, welcher Grad das Polynom der O-Notation aufweist. Im Gegensatz dazu sind Algorithmen der Komplexitätsklasse NP solche, die eine nicht-deterministische polynomielle Laufzeit haben, also beispielsweise oft erst nach permutierenden $O(n!)$ zu einem exakten Ergebnis kommen – was in diesem Beispiel hier noch schlimmer ist als die altbekannte exponentielle Laufzeit und schon bei Größen von um die $n = 10$ die Laufzeit explodieren lässt – dafür aber die sich ergebende Lösung in polynomieller Laufzeit überprüft werden kann. Nun hat sich in der

Geschichte der Mathematik gezeigt, dass es eine Liste von sogenannten NP-vollständigen Problemen gibt, in die sich prinzipiell auch andere Probleme umformen lassen – und die, wenn man nur eines dieser Probleme lösen würde, zeigen würden, dass $P = NP$ ist, also die Klasse der Probleme mit polynomieller Laufzeit die selbe Klasse an Problemen ist wie die der nicht-deterministischen polynomiellen Laufzeit. Das TSP ist eines dieser NP-vollständigen Probleme, wie im ersten Absatz dieses Blogartikels schon erwähnt. Kurz zusammengefasst geht es beim TSP um einen fiktiven Handlungsreisenden, der das Ziel hat die kürzeste Rundreise von einer Stadt aus zu einer Zahl n anderer Städte zu wählen und anschließend zum Ausgangspunkt zurückzukehren.

Kurze Erklärung zu den hier im Artikel häufig verwendeten Begrifflichkeiten: Die Effizienz eines Algorithmus meint in unserem Fall die Laufzeit des Algorithmus. Wenn ein Algorithmus also effizient ist, hat er eine vergleichsweise geringe Laufzeit und gemünzt auf das P-NP-Problem im besten Fall eine maximal polynomielle Laufzeit. Die Effektivität eines Algorithmus meint in unserem Fall, dass er mit der Ausgabe / mit dem Output einer exakten Lösung endet und terminiert – anhand unseres TSP also, dass er die kürzeste Rundreise einer Städtetour / den kürzesten Hamiltonkreis eines Graphen zurückliefert. Wir werden in den folgenden Beispielen und Implementierungen sehen, dass zumindest einer der beiden Faktoren nicht gegeben ist für gängige algorithmische Lösungsansätze und das TSP daher also nicht so trivial zu lösen wie man intuitiv glauben würde.

/tspiteration/01-iterate-tsp.py ist meine erste naiv implementierte und logisch skalierbare – aber nicht laufzeitskalierbare – exakte Lösung für das TSP mit hauptsächlich einem Brute-Force-Ansatz und verschiedenen Listenoperationen – geschrieben in Python. Die asymptotische Laufzeit wäre $O(n!)$, was in Bezug auf die Effizienz keine Lösung darstellt und damit weit von einer Lösung für das gesamte P-NP-Problem entfernt ist.

/tspiteration/02-greedy-tsp.py ist mein zweiter implementierter algorithmischer Ansatz, ebenfalls eine logisch skalierbare und diesmal auch laufzeitskalierbare – wenn auch nicht exakte – Lösung für das TSP mit hauptsächlich einem Greedy-Ansatz – geschrieben in Python. Die asymptotische Laufzeit wäre $O(n^2)$, was hinsichtlich der Effizienz eine polynomielle Lösung darstellt, jedoch keine effektive Lösung für das gesamte P-NP-Problem ist.

/tspiteration/03-compare-tsp.py ist mein dritter implementierter algorithmischer Ansatz, ebenfalls eine logisch skalierbare und diesmal auch laufzeitskalierbare – wenn auch nicht exakte – Lösung für das TSP mit hauptsächlich einem Greedy-Ansatz – geschrieben in Python. Die asymptotische Laufzeit beträgt $O(n^2)$, was zwar hinsichtlich der Effizienz eine polynomielle Lösung darstellt, jedoch keine effektive Lösung für das gesamte P-NP-Problem ist. Die Besonderheit dieses Ansatzes ist eine zusätzliche Vergleichsoption.

/hypothesis ist eine Materialsammlung zu einer neuen algorithmischen Idee und hypothetischen alternativen Betrachtungsweise des Traveling Salesman Problems. In diesem Unterordner gehe ich der Spur nach, ob man das TSP vielleicht gar nicht zwangsläufig als ein Problem der Graphentheorie betrachten sollte, sondern vielleicht eher als ein Problem der algorithmischen Geometrie, da es sich bei den im Problem beschriebenen Städten oder allgemein physischen Orten ja nicht zwangsläufig um einen Graphen nach Graphentheorie handelt – mit seinen klassischen Eigenschaften der Ortsunabhängigkeit in der Darstellung – auch wenn Graphen ein bisher sinnstiftendes Modell zur Darstellung und Modellierung von Netzwerken (wie beispielsweise Straßennetzen oder Seewegen) sind und historisch wegen ihres Ursprungs im Eulerkreis als solche gewachsen sind.

Dennoch wage ich hierin mal eine meines Wissens und Gewissens nach neue, mindestens aber andere Betrachtungsweise des TSP und versuche meinen Ansatz zu formalisieren und zu beschreiben. Ganz allgemein gehe ich davon aus, dass man zur eventuellen Lösung des TSP im Rahmen des P-NP-Problems annehmen sollte, dass sich die Knoten des Graphen gemäß ihrer maßstabsgetreuen Ortsrelationen – der Einfachheit halber zunächst auf den zweidimensionalen Raum übertragen – unbeweglich darstellen lassen sollten. Dies erreiche ich mit Kennzeichnung der Punkte an der sich die Knoten des Graphen befinden sollten auf einem klassischen Koordinatensystem, wie man es in der linearen Algebra und Geometrie kennt. Kennzeichnend sind hier die X- und die Y-Achse.

Dieses Wissen angenommen, würde das beispielsweise bedeuten, dass man mit der klassischen Adjazenzmatrix, die man beim Programmieren beispielsweise verwendet um Graphen und ihre Relationen abzubilden, nicht zur Lösung des TSP kommen würde – und dass es bisher nicht in polynomieller Laufzeit gelöst wurde, lässt dies zumindest vermuten – auch wenn es dennoch einige exakte Lösungen zu diesem Problem gibt und man dafür die bisherige Graphenmodellierung gut verwenden konnte, wenn auch nicht in polynomieller Laufzeit. Da man durch die Projektion des TSP auf einen Graphen nach Graphentheorie hypothetisch bestimmte für die Problemlösung relevante Informationen verlieren könnte, gehe ich persönlich davon aus, dass dies eine Ursache für die bisher vergebliche Suche nach der Lösung sein könnte. Das TSP als Problem der algorithmischen Geometrie zu betrachten, würde beispielsweise trigonometrische

Berechnungen oder zirkulare Berechnungen ermöglichen, als auch Berechnungen des Winkels u.v.m. Meines Erachtens liefert die geometrische Betrachtung einen ganz neuen Baukasten an Möglichkeiten, um das Problem anzugehen. Das ist also so im Großen und Ganzen mal meine – bisher unbestätigte – Vermutung dazu.

Update und Nachtrag (Stand: 17. Dezember 2025) zur geometrischen Betrachtungsweise

des TSP: Tatsächlich bewege ich mich mit der Betrachtung und Hypothese, die ich obenstehend in den Absätzen geschildert habe, auf bereits ausgetrampelten Pfaden der Mathematik. Das TSP alternativ als Problem der Geometrie zu betrachten, ist lange schon in der Mathematik verankert und bekannt und ist in diesem Zusammenhang unter dem Namen des euklidischen TSP oder auch des metrischen TSP bekannt. Dabei ändert sich nichts daran, dass es weiterhin ein insgesamt NP-vollständiges Problem ist. Dennoch freut mich, dass meine Annahmen parallel zur graphentheoretischen Betrachtungsweise korrekt sind und ich auch selbstständig zu der Erkenntnis kam, auch wenn es natürlich längst bekannt war. Gleichzeitig überrascht es mich nicht, da es in der Welt der Mathematik weitaus schlauere und fähigere Menschen gibt als mich.

Ein algorithmischer Ansatz, den ich hier anhand meiner Annahmen versuche, ist der – wie ich ihn nenne – Vakuumierungsalgorithmus. Auf wenige Worte heruntergebrochen kann der Algorithmus dem Titel nach wie folgt beschrieben werden: Wir betrachten einen Graph als mit fest im Raum verankerten und fest im Raum stehenden Knoten entsprechend ihrer maßstabsgetreuen Ortsrelationen zueinander. Wir konstruieren rund um die Knoten eine virtuelle zirkulare Form / einen virtuellen Kreis / eine virtuelle Ellipse, sodass sich alle Knoten des Graphen innerhalb der zirkularen Form befinden. Anschließend "vakuumieren" wir die virtuelle zirkulare Form unter der Annahme, dass diese sich vergleichsweise elastisch verhält – verformen sie also insofern als dass sie sich bis zum engsten Grad um die Knoten zusammenzieht und sich insgesamt zu Verbindungslinien der Knoten abstrahiert. Für einfachere eher konvexe Formen von Ursprungsgraphen ließ sich der Vakuumierungsalgorithmus sehr schön graphisch beweisen und simulieren – aber mehr zur Rolle der Simulationen zu einem späteren Zeitpunkt.

Mit dem hier gelungenen Beweis kann zumindest festgestellt werden, dass der Vakuumierungsalgorithmus eine Teillösung zum TSP für spezifische Formen von Graphen ist. Da ein potenzieller Lösungsalgorithmus natürlich auch Funktionen zur vorherigen Analyse des Graphen haben kann sowie im Zweifelsfall auch eine Aneinanderreihung von if-elif-else-Statements oder auch Switchcases, kann in diesem Moment noch nicht ausgeschlossen werden, dass er eine Rolle spielt.

Update und Nachtrag (Stand: 18. Dezember 2025) zu meiner Version des

Vakuumierungsalgorithmus: Tatsächlich bewege ich mich auch hier mehr oder minder auf bereits beschrittenen Pfaden der Mathematik und ganz konkret der algorithmischen Geometrie und algorithmischen Technik. In diesem Sinne ist der Vakuumierungsalgorithmus lediglich eine Abwandlung der Delaunay-Triangulation in Kombination mit einem Shrinking-Ansatz. Beides bewährte, aber eben leider auch schon lange bekannte Techniken, sodass ich prinzipiell auch hier nichts Neues zur Forschung beigetragen habe. Wirkliche Innovation ist eben leider nicht immer so einfach wie gedacht – auch wenn es seit dem Informationszeitalter zumindest leichter ist mögliche Innovationen schneller als keine solchen zu erkennen, wie in unserem Beispiel hier veranschaulicht. Die asymptotische Laufzeit für die Delaunay-Triangulation beläuft sich einer kurzen Recherche nach insgesamt wohl etwa auf $O(n \log n)$, sodass ich davon ausgehe, dass sich

die hier als Vakuumierungsalgorithmus geschilderte Version auch in etwa in dieser Komplexitätsklasse befinden wird. Danke an dieser Stelle für den Hinweis zum Shrinking-Algorithmus und zu Heuristiken an meine Professorin für Mathematik.

So euphorisch wie mich der erste Versuch mit dem Vakuumierungsalgorithmus gestimmt hatte, so nüchtern hat es mich zurückgelassen, als es vergleichsweise einfach auf Anregung meiner Professorin für Mathematik und meines Professors für theoretische Informatik gelungen ist, mit einem etwas komplexeren Graphen den Vakuumierungsalgorithmus für bestimmte Formen und Anordnungen von Knoten zu falsifizieren. Hier angenommen wird ein Graph, dessen Aufbau insgesamt rubinförmig / kreisförmig ist und der Knoten in zweireihiger Anordnung aufweist, insgesamt zwölf an der Zahl. Angenommen wir setzen die Suktion für das zugrundeliegende Vakuum immer mittig zur zirkularen Form, dann kommen wir im Falle dieses bestimmten Graphen schon nicht mehr auf eine effektive und exakte Lösung – und auch die Suktion anders zu platzieren führt wahrscheinlich nicht zu einer optimalen Lösung, ist aber im Zweifelsfall auch ein Parameter. Der Algorithmus terminiert in der Abstraktion zu einer Sternform, obwohl der kürzeste Weg zwischen den Knoten eher wie die Silhouette eines Gespenstes aussieht. Ich nenne diese Antithese daher "Christmas ain't Halloween".

Nachdem mir in der Folge das bloße Philosophieren darüber und die rein graphische Herangehensweise auf Grundlage meiner eigenen Imagination und die Limitation meiner eigenen mathematischen Fähigkeiten nicht ausgereicht hat, den Vakuumierungsalgorithmus zu untersuchen, bin ich in der Folge dazu übergegangen am selben Wochenende der Ideenfindung ein bisschen zu "vibecoden" – wie man es heute wohl in der Praxis macht. Entstanden ist mit der Hilfe von Microsoft Copilot ein kleines physikalisches Simulationstool, mit dem man den Vakuumierungsalgorithmus – wie hier in der Hypothese beschrieben – ausprobieren kann. Ich habe vor, mit meinem Simulationstool mal ein paar Experimente zu wagen und weiter darüber zu brüten unter welchen Gesichtspunkten man das TSP vielleicht noch irgendwie lösen kann – oder eben auch nicht, da auch ein Gegenbeweis die Frage des P-NP-Problem lösen würde.

Abgesehen davon ist es mir natürlich ein Anliegen meine eigenen Fähigkeiten zu schärfen, um für kommende Fragen vorbereitet zu sein und um im Zweifelsfall meine Ideen dazu plausibel mathematisch darzulegen – im besten Falle natürlich in Form von Beweisen. Ein Buch über die Grundlagen der algorithmischen Geometrie aus der Hochschulbibliothek liegt quasi metaphorisch schon unter meinem Kopfkissen und wartet darauf in den Semesterferien gelesen zu werden.

Und auch, wenn ich nach der Falsifizierung eigentlich mit dem Philosophieren über den Vakuumierungsalgorithmus durch war, bin ich natürlich nochmal in den kurzen Austausch mit meinem Professor für theoretische Informatik nach einer Vorlesung gegangen, über meine persönlichen neuesten Erkenntnisse zum P-NP-Problem, aber auch das Simulationstool – der in der Folge doch noch zu einer tatsächlich mathematisch-philosophischen Frage geführt hatte – aber das nur am Rande und vielleicht führt das auch zu weit. Mein Professor erwähnte in unserem Dialog, dass Sprichwort "A scientist is a fool. He tries to prove things once, twice, three times, a thousand times - but never a thousand and one times" und versuchte mir anhand dessen wahrscheinlich zu erklären, dass es in der Wissenschaft nicht zwangsläufig vergeblich sein muss, etwas auch über die "gesunde" Zahl an Versuchen hinaus, gemessen an der etwaigen Kosten-Nutzen-Rechnung, zu probieren / zu testen / zu versuchen. Offenbar ein Mutzuspruch, nach den letzten misslungenen Versuchen das TSP im Hinblick auf die polynomielle Laufzeit zu optimieren. Trotzdem beschäftigte mich das Zitat noch ein Weilchen danach...

...denn der Hintergrund, warum das P-NP-Problem ein so dopamintreibendes Problem ist, beruht ja letztendlich auch auf dem Ansatz des Clay Mathematics Institute, das aufgrund der Wichtigkeit der Lösung des Problems öffentlich ein Preisgeld ausgerufen hat, im Rahmen der Ernennung zum Millennium-Problem – und damit die Lösung des Problems gleichzeitig in Aussicht auf alle hochdotierten wissenschaftlichen Würden gestellt hat, die es in diesem Bereich wohl geben kann. Demjenigen, der das P-NP-Problem zu lösen imstande ist, winkt neben einer runden Millionen US-Dollar Preisgeld sehr wahrscheinlich auch die Fields-Medaille (das Äquivalent zum Nobelpreis in den Sphären der Mathematik). Getreu dem Motto "Throw more assets at the problem", ist das ein Einsatz um möglichst viele schlaue Köpfe zu motivieren, sich mit dem P-NP-Problem und generell mit den NP-vollständigen Problemen (davon gibt es ja gleich mehrere) zu beschäftigen.

Frühe Formen des TSP gibt es seit 1800 und schlussendlich Einzug in die große akademische Welt der Mathematik hatte es in den 1930ern gehalten und damit im Verlauf auch in die Informatik. Die mathematisch-philosophische Frage, die sich mir in der Folge jedoch stellt ist diese: Wenn seit den 1930er Jahren jede*r Professor*in oder mindestens Dozent*in der Mathematik oder theoretischen Informatik jedes Semester oder auch jedes Jahr einen Kurs von – sagen wir mal mindestens 10 – bis maximal ein vollbesetztes Auditorium (Größe abhängig von den

Räumlichkeiten) an Studenten auf dieses Problem aufmerksam gemacht hat, um es zu lösen, ist das dann nicht bildlich gesprochen mindestens ein Ansatz mit exponentiellem Wachstum derjenigen Menschen und schlaun Köpfe, die das Problem kennen und zumindest einmal versucht haben es zu lösen – also eine asymptotische Laufzeit von $O(2^n)$ oder gemessen am "Throw more assets at the problem"-Ansatz des Clay Mathematics Institute auch eine Art Brute-Force-Ansatz mit asymptotischer Laufzeit von $O(n!)$? Und eine viel wichtigere Frage in der Folge: Wäre von dieser Metaebene aus betrachtet – selbst wenn jemand zu einem Beweis käme, dass $P = NP$ ist – dieser Beweis, dass eine polynomielle Laufzeit für einen Lösungsalgorithmus (von dem wir derzeit noch nicht wissen, dass er existiert), nicht hinfällig – gemessen daran, wie lange es tatsächlich gebraucht hat zu diesem Beweis und letztendlich zu einer polynomiellen Problemlösung zu kommen?

Ich lasse diese Frage jetzt einfach mal so im Raum stehen und verabschiede mich erstmal in die Wochen der Klausurvorbereitung zum hoffentlich erfolgreichen Abschluss meines ersten Semesters im Intensivstudiengang Informatik.

Zum Beweisversuch in **/mathematics/01-attemptedproof-tsp.pdf** hat mich unter anderem ein Gespräch mit einem befreundeten Wissenschaftsforscher und Designer inspiriert, der zu mir meinte, dass man im Zweifelsfall allgemein in der Wissenschaft und Forschung auch versuchen kann zu zeigen, dass etwas nicht so ist wie angenommen (also Antithese und Falsifikation aufzustellen), aber auch der Unterricht / die Vorlesungen an der Hochschule für Wirtschaft und Recht Berlin bei meiner Professorin für Mathematik, in welchem wir das Beispiel von Hilberts Hotel im Hinblick auf verschiedene Größen von Unendlichkeit behandelt haben und in diesem Zusammenhang Aleph-Null behandelt haben, als auch Cantors Diagonalverfahren, die allgemeine Mengenlehre sowie vieles Weitere aus Algebra und linearer Algebra. Der Unterricht / die Vorlesungen an der Hochschule für Wirtschaft und Recht Berlin bei meinem Professor für theoretische Informatik haben mich darüber hinaus inspiriert, mich mit dem P-NP-Problem und dabei speziell mit dem TSP auseinander zu setzen, auch hier sind wir Cantors Diagonalverfahren nochmal durchgegangen, als auch haben wir uns vieles Weitere aus der Logik, aber auch Algorithmen und Datenstrukturen angesehen wie zum Beispiel auch die Definitionen von asymptotischer Laufzeit und ihren Abhängigkeiten zur Eingabe. Darüber hinaus haben mich natürlich auch meine eigenen fehlgeschlagenen Versuche das TSP in polynomieller Laufzeit zu lösen inspiriert, aber auch das parallele Prompting mit Microsoft Copilot und ChatGPT über das Problem immer wieder mal und meine Möglichkeit mit diesen Systemen über das P-NP-Problem aber auch das TSP zu kommunizieren. Für die schriftliche Formalisierung meiner Ursprungsidee hat mir insbesondere die Synergie eines Chats mit ChatGPT geholfen und meine Rückfragen zur Abzählbarkeit von Mengen dort, aber auch die Recherchen zu vorher benannten Themen mit Google und Wikipedia. Die zugrundeliegende Ursprungsidee hatte ich jedoch beim klassischen Grübeln über das Problem während des Einschlafens in der Nacht vom 18. Dezember auf den 19. Dezember 2025 und grundlegend kam mir der Gedanke, dass es sich anhand der durch das TSP definierten Mengen und Eingaben um ein unendlich komplexes Problem handeln müsste und es dadurch nicht zu einer Lösung in polynomieller Laufzeit für alle Eingaben n kommen kann und daher auch bislang lediglich zu heuristischen Lösungsversuchen gekommen ist, in der Zeit in der das Problem existiert und der Allgemeinheit bekannt ist, da man aufgrund der Kardinalitäten der verschiedenen Mengen immer eine noch größere Eingabe n finden kann. Das versuche ich mit meiner Rechnung zu beweisen und letztendlich mit meiner vergleichbar kurzen Rechnung

plausibel und nachvollziehbar darzulegen. Ich versuche mal meine Gedanken zu erklären – eine erhebliche Schwierigkeit dabei ist, dass meine Gedanken dazu visueller Natur sind.

Die Eingabe, die ich in einen Algorithmus vornehme, wird bei der Berechnung der asymptotischen Laufzeit immer durch n angegeben. In unserem Fall ist n eine natürliche Zahl aus $\mathbb{N}_0$ und entspricht der Anzahl der Punkte, die ich in einen Algorithmus zur Lösung des TSP geben würde. Ein einziger Punkt wird durch $n = 1$ definiert und entspricht den Koordinaten (x, y) , wobei x und y aus der Menge der reellen Zahlen $\mathbb{R}$ stammt. Abgebildet werden diese Punkte auf einer unendlichen und zweidimensionalen Ebene des $\mathbb{R}^2$ – insofern werden bereits die natürlichen Zahlen aus $\mathbb{N}_0$ auf die Menge der reellen Zahlen $\mathbb{R}$, ferner des $\mathbb{R}^2$ abgebildet. Dadurch gibt es bereits eine erste Kollision der Kardinalitäten. Die Kardinalität der natürlichen Zahlen $|\mathbb{N}_0|$ ist Aleph-Null, also $|\mathbb{N}_0| = \aleph_0$, die kleinste Kardinalität aus einem Zahlenraum der als abzählbar gilt. Die Kardinalität der reellen Zahlen $|\mathbb{R}|$ ist insgesamt $2^{(\aleph_0)}$ und in der Schreibweise des Beweisversuches als $|\mathbb{R}| = 2^{(\aleph_0)} = \mathfrak{c}$ ausgedrückt. Also ist $\mathbb{R}$ ein Zahlenraum höherer Kardinalität als $\mathbb{N}$ – und $\mathbb{R}$ gilt daher bereits als überabzählbar. So weit so gut – doch wirklich spannend wird es erst, wenn wir uns die Menge aller Punkte P aus der Eingabe n ansehen und letztendlich deren Gesamtheit aller Ortsrelationen ansehen, die ich als tatsächliche Eingabe in den Algorithmus identifiziere und die ich als Potenzmenge der Punktmenge definiere, im Beweisversuch dargestellt als $\mathcal{R} = \mathcal{P}(P \times P)$. Wenn wir ein größtmögliches n annehmen, das aufgrund der Kollision der Kardinalitäten schon gar nicht existieren kann, dann wäre die Kardinalität der Menge aller Ortsrelationen bereits $|\mathcal{R}| = 2^{(2^{(\aleph_0)})} = 2^{\mathfrak{c}}$ – was als stark überabzählbar gilt. Die Eingabe n wäre hierbei eine unendliche Menge an Punkten, die sich zu einer Fläche verdichten. Aufgrund der Hierarchie der Kardinalitäten $\aleph_0 < \mathfrak{c} < 2^{\mathfrak{c}}$ komme ich persönlich also zu dem Schluss, dass die Komplexität des Problems durch das Kontinuum der endlichen Eingabe n bedingt ist und damit die Problemstellung aus dem euklidischen TSP allein durch das im Problem Gegebene unendlich komplex werden kann, da ich immer eine noch größere und noch komplexere Eingabe n finden kann. Folglich kann es wahrscheinlich keine viel andere und wesentlich kürzere und gleichzeitig exakte algorithmische Lösung für alle n außer die des Brute-Force-Ansatzes mit der altbekannten

permutierenden asymptotischen Laufzeit von $O(n!)$ geben, die bei einem regulären TSP nach Graphentheorie die Zahl der vom Algorithmus betrachteten Ortsrelationen auf eine Adjazenzmatrix mit paarweise betrachteten Abständen zwischen den Knoten beschränkt – und daher war es wahrscheinlich auch in der 100- bis 200-jährigen Geschichte des TSP kaum möglich andere Lösungen für das TSP zu finden, die nicht lediglich Heuristiken sind oder nicht mindestens exponentielle Laufzeit haben – und das für vergleichsweise kleine Eingaben – zumindest klein im Vergleich mit ∞ . Dadurch, dass das euklidische TSP meinen Recherchen nach auch zu den NP-vollständigen bzw. -harten Problemen zählt, komme ich daher auch zu dem Schluss, dass $P \neq NP$, da man für dieses NP-vollständige Problem aufgrund der unendlichen Komplexität des Gegebenen und dadurch der Größenrelationen der tatsächlich möglichen Eingaben n niemals zu einem Algorithmus mit polynomieller Laufzeit für speziell dieses Problem kommen wird. Ich denke es liegt im Naturell des Problems. $\square$?

Realistische Betrachtungsweise (Stand: 24. Dezember 2025) meines formulierten

mathematischen Beweisversuches: Genau genommen ist es insgesamt sehr sehr unwahrscheinlich, dass ein beinahe Erstsemester der Informatik (ganz konkret Erstsemester, wenn man bestandene Onlinekurse über informatische Themen wie das Programmieren und absolvierte Brückenkurse außen vorlässt – was näher an der Realität ist, da ich keine Anrechnung dieser Leistungen beantragt habe) das als ganze betrachtete P-NP-Problem löst – daher habe ich meinen Beweisversuch auch mit einem dicken Fragezeichen versehen. Vermutlich ist der hier obenstehende Beweisversuch und seine formellen Ausdrücke nur eine psychische Projektion dessen, dass die bisher im Informatikstudium beleuchteten Themen für einen Erstsemester erstmal wirklich unendlich komplex wirken. Formell betrachtet wird mein Beweisversuch einige Fehler haben, derer ich mir mit meinem begrenzten Verstand gar nicht bewusst sein kann, da ich sehr sicher mit meinem angefangenen ersten Semester keine Vollständigkeit auf die höhere Mathematik erheben kann. Vielleicht – aber nur vielleicht – ist meine Betrachtungsweise in irgendeinem Feld der Informatik und Mathematik an tatsächlichen Forschungslinien und daher in irgendeinem Kontext relevant – aber welche das sein könnten, das ist mir aktuell nicht bewusst. Wahrscheinlich wollte ich mir nur selbst eine gedankliche Grenze setzen, an der ich nicht weiter komme und damit die Betrachtung erstmal auf den Ablagestapel legen, anstatt mich weiter darin

zu verrennen. Trotzdem spricht dieser Blogartikel als Gesamtes betrachtet im besten Falle für meine Problemlösekompetenz, ein ungelöstes Problem sehr genau und eingängig zu betrachten, bevor man versucht realistisch zu entscheiden, dass es für einen selbst unlösbar ist oder zumindest unlösbar erscheint. Dieses Urteil überlasse ich erstmal dem*der Leser*in des Artikels und ich versuche mich auf Ratschlag meiner Professorin für Mathematik erst wieder im vierten Semester mit dem TSP auseinander zu setzen, an einer Stelle im Bildungsverlauf, in der sich unser Studienkurs nochmal eingängiger mit dem Problem befassen wird.

Post Scriptum (Stand: 14. Mai 2026): In einem weiteren Gespräch mit meiner Professorin für Mathematik über dieses Thema, meinte diese zu mir, dass es Phänomene wie oben in meinem Beweisversuch beschrieben, auch unter der Bezeichnung der sogenannten "Linearen Regression" gibt. Lineare Regression bezeichnet umgangssprachlich die Abhängigkeit von Variablen und deren Komplexität. Vielleicht sind wir mit dem Beweisversuch also doch etwas näher an der Lösung? Wiederum ein Artikel der österreichischen Zeitung DerStandard mit der Überschrift "Unendlich ist doch nicht gleich unendlich" vom 19. August 2019, den ich in der Folge meines Blogartikels vom Algorithmus empfohlen bekommen und gelesen hatte und der sich mit den Forschungsergebnissen der TU Wien zu Kardinalitäten und letztendlich der Abzählbarkeit von Unendlichkeiten befasst, wirft neue Fragen zur Beweisbarkeit von Kardinalitäten im Allgemeinen auf. Ich meine aber vereinfacht gesagt damit recht zu behalten, dass zwischen zwei Punkten bzw. zwischen Punktpaaren im Koordinatensystem, unter der Annahme, dass die Punkte selbst unendlich klein sind, immer noch ein weiterer Punkt gefunden werden kann, der das Problem selbst unendlich verkomplizieren kann.